

Aashan — Product and Architecture Opinion

After reading the repository and `expense_tracker.xlsx`

2 September 2026 · nothing modified

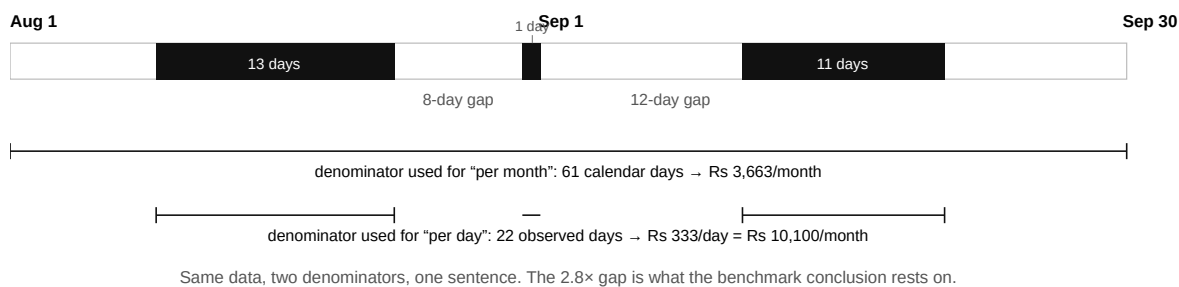
The workbook changed my view of this project more than the code did. Three things fell out of it that I did not expect, and they reframe most of the ten questions.

Discovery 1 — The spreadsheet's most confident conclusion is backwards, and the cause is the thing Aashan must model

The Insights sheet says day-to-day food is “about Rs 333/day or Rs 3,663/month”, and the Kochi sheet compares that ₹3,663 against a ₹6,500 typical student food budget, concluding the habits are “on the leaner end of typical”.

Those two numbers use different denominators. ₹333/day is $7,326 \div 22$ days on which a meal was recorded. ₹3,663/month is $7,326 \div 2$ calendar months. They are presented in one sentence as equivalent. They differ by a factor of 2.8. At the observed rate of ₹333/day, eating every day costs about **₹10,100/month, not ₹3,663** — which is well *above* the ₹6,500 benchmark, not below it.

Every “per month” figure in the workbook is $\text{total} \div 2$, and every “per day” figure is $\text{total} \div \text{active days}$. The headline “Aug 1 to Sep 30” covers 61 days; the data covers **25 days**, inside a 43-day span, with an 8-day and a 12-day hole.



This is not a spreadsheet mistake to be corrected. It is the single strongest argument for a concept the current schema does not have: coverage. Any rate — per day, per month, average, “compared to last month” — is meaningless without knowing which days were actually observed. Automating ingestion does not make this go away; it moves it. A bank statement covers a known window, SMS covers from the day the app was installed, AA covers whatever the consent granted, and cash covers nothing. Aashan will always be reasoning over partial coverage, and unless coverage is a first-class dimension it will publish the same class of confidently-wrong number, just faster.

Discovery 2 — Aashan's three ingestion sources cannot produce this spreadsheet.

93 transactions. Of those, **87 (94% by count, 58% by value)** are between ₹20 and ₹110: breakfast, lunch, dinner, coffee, a ₹65 water can every second day. Six transactions carry the other 42%:

2026-08-09	Household Items	1,740.00	apartment setup
2026-08-09	Groceries	1,605.00	grocery restocking
2026-08-29	Transport/Fuel	974.71	petrol, full tank
2026-08-18	Subscriptions	400.00	ChatGPT Go
2026-08-18	Hackathon	348.00	domain purchase <- the Rs 348 in your example
2026-08-21	Festival	250.00	Onam fund

Those six would appear cleanly in a statement, an AA feed and an SMS alert. The other 87 are the ones that are cash, or a UPI transfer to a mess owner's VPA that arrives as UPI/9847xxxxxx@vbl/PAYMENT with no hint that it was lunch. **The**

category that dominates the workbook and drives every one of its insights — 58% of spend — is the category all three automated sources handle worst.

Two consequences. First, manual/cash entry is not a fallback source; on this evidence it is the *primary* source for the majority of transaction count, and it needs a genuinely fast capture path, not a form. Second, the semantic granularity the workbook depends on — breakfast versus lunch versus dinner, “new apartment setup”, “hackathon project” — is human-authored intent that no bank feed carries and no classifier can recover. Aashan can infer *merchant* and often *category*. It cannot infer *why*. If the product’s value depends on knowing why, it needs a cheap way to ask.

Discovery 3 — The spreadsheet is not a dashboard, and neither should Aashan be

Four of the five sheets are raw material. The product is the fifth: “Why the total looks high.” It takes ₹12,644, and instead of charting it, it argues about it — ₹3,345 of that is a one-time move-in, strip it and the picture inverts; the food number is large because it repeats daily, not because any meal is expensive; here is what is cuttable and here, explicitly, is what is not.

That is a different product from the one in the repository. Both existing frontends render totals, category shares and trends. None of them explains anything. **The spreadsheet’s implicit brief is an explanation engine with a ledger underneath, not a dashboard with insights bolted on.**

1 · Product vision

What is genuinely valuable

The canonical pipeline is right, and it is the hard part most personal-finance projects skip. Separating lifecycle from classification from budget-inclusion from transfer state, keeping provenance, insisting corrections beat automation — that is the correct spine and it already exists in the schema. Keep it.

The reconciliation problem is real, and it is the actual moat. Anyone can parse a statement. Very few products can tell you that the ₹348 your bank shows on the 20th is the same ₹348 your SMS showed on the 18th, and not two purchases. Getting that right is worth more than any AI feature on the roadmap.

Privacy-first is a genuine differentiator in Indian personal finance, where the category norm is to sell the data. But see section 9 — the current version of the claim is not the one to defend.

What I think is over-built

Three sources is really two information classes. PDF/CSV and AA deliver the same thing: a settled, authoritative record of what the bank says happened, arriving after the fact. AA is a nicer pipe — no upload, no password, structured — but it carries no information a statement does not. SMS is the only source that carries something genuinely different: latency. It tells you about a transaction within seconds.

Framed that way, the priority ordering falls out, and it is not the one in the roadmap. **AA is the lowest-marginal-information, highest-cost source of the three**, and it is the one with a 5–10 month regulatory blocker attached. It should be last. I said this in the reconnaissance; the workbook makes it stronger, because AA would have captured six of the ninety-three transactions.

“The user should see that a transaction is supported by multiple sources” is a gimmick. This is on your list and I think it is wrong. Nobody wants a confidence badge on their lunch. Multi-source evidence is essential *internally* — it is how you avoid double-counting and how you resolve conflicts — but as a user-facing feature it is engineer-fascination. What users actually need from reconciliation is two things: don't show me the same purchase twice, and tell me when a number is incomplete. Surface *coverage* and *unresolved conflicts*. Do not surface a provenance chip on every row.

Goals, benchmarks and multi-currency are all premature. Nothing in the workbook needs a goal. The benchmark is worse than premature — see section 2.

2 · Spreadsheet → product

Preserve

1. **Decomposition of a headline number.** “₹12,644, of which ₹3,345 is one-time” is the highest-value output in the file. Recurring baseline / one-off / exceptional, computed deterministically, is the core intelligence feature.
2. **Normalisation by an honest denominator.** Per-day and per-month rates, with the denominator stated. The workbook gets this wrong, which is exactly why it belongs in software.
3. **Exceptional-spend attribution.** Not “here is an anomaly” but “this specific ₹1,740 on 9 August is 14% of your two months”. Attribution beats detection.
4. **Counterfactual arithmetic.** “Cut coffee and snacks entirely and you save ₹140/month.” This is a computable, checkable statement and it is the most actionable thing in the file.
5. **Explicit non-recommendations.** The sheet says the Onam fund and the hackathon domain are not really cuttable and excludes them from the savings estimate. Refusing to give advice is a feature. Most finance apps would have told him

to cut both.

6. **The user's own vocabulary.** He invented “Hackathon”, “Festival”, “Water”, “Household Items”. None of these are in Aashan's nine built-in categories. User-defined categories are not a nice-to-have.

Improve

- **Every number becomes clickable.** “26% of the entire two-month total” should open the two transactions behind it. The spreadsheet asserts; the product should let you check.
- **Recurring should be detected, not hand-declared.** The sheet's recurring/one-off split is a human judgement encoded in a SUMIF. In the product it should be a detected series with a cadence and a next-expected date — which then also powers cash-flow forecasting and “this subscription went up” alerts.
- **Show the denominator, always.** Any rate should be rendered with its coverage: “₹333/day across 22 observed days” — never “₹333/day”.
- **The category column is three dimensions wearing one hat.** “Food – Breakfast” is category + meal slot. “Hackathon” is a project context. “Festival” is an occasion. These want to be *category* plus *free-form tags*, which is cheap to build and matches how he actually thinks.

Discard

The Kochi comparison. Entirely. It is built on cost-of-living blog posts, is self-admittedly “assumptions, not exact figures”, and — as shown above — its headline conclusion is an artifact of a denominator error. Comparing one person's partial-coverage spend against a stranger's blog-derived cohort average produces a number that feels authoritative and means nothing. Do not build a `benchmark` entity. If you ever want peer comparison, the only honest version is the user against their own past, which needs no external source and no cohort assumptions.

The lifestyle advice, as product output. “Cook breakfast and dinner at home”, “use the bus for short trips”, “the free tier may be enough” — this is life coaching, not financial analysis. It is also the part most likely to be handed to an LLM and the part most likely to land as patronising. Keep the arithmetic (*if* this category went to zero, you save ₹X). Drop the instruction.

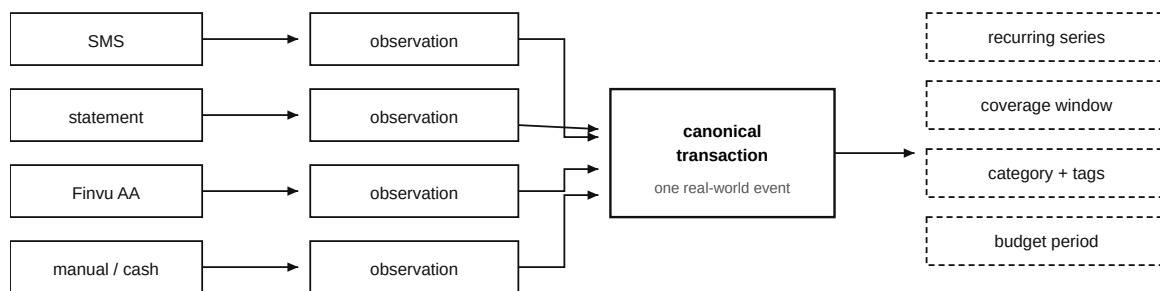
The “average day” framing where coverage is partial. Until Aashan knows which days it can see, it should refuse to publish a daily average rather than publish a wrong one.

3 • Financial data model

Taking the instruction seriously: only what must exist in the backend, and why.

Concept	Verdict	Reasoning
Transaction	Entity — exists	Canonical, one row per real-world financial event. Keep the separated state dimensions.
Source observation	New entity — the key change	Today a transaction has one <code>source</code> column. With three sources that is wrong: the same event is <i>observed</i> by SMS, then by a statement, then by AA. Make it <code>transaction 1:N observation</code> , each carrying source, provider, external ID, observed-at, the normalised fields as that source reported them, and the import it came from. Deduplication then becomes “attach an observation to an existing transaction” instead of “drop the row”, which is what makes conflict handling, provenance and overlapping-statement reingestion all work with one mechanism. Without this, everything in section 4 is a special case.
Coverage window	New entity	<code>(account, source, from, to, completeness)</code> . Makes every rate honest, makes “what am I missing” answerable, and is the only defence against Discovery 1. Cheap to build, and nothing else provides it.
Account	Entity — exists, unused	Reconciliation is per-account (two accounts can legitimately have identical ₹348 on the same day). Coverage is per-account. Transfer detection needs two accounts. Currently <code>account_id</code> is nullable and nothing populates it. Must become real before SMS lands.
Recurring series	New entity	Not a boolean on a transaction. A series has a merchant, a cadence, a typical amount, a next-expected date, and member transactions. It powers the workbook's central insight, cash-flow forecasting, and price-increase alerts. This is the highest-value new entity after observation.
Spending nature	Derived dimension	recurring / one-off / exceptional. Computed from series membership and amount distribution, stored for stability, recomputable. Not an entity.
Category	Entity — exists, bypassed	The table exists; transactions store <code>category_name</code> as free text and <code>category_id</code> is never set. User-defined categories are required by the workbook's own vocabulary. Fix the bypass.
Tag / context	New, small	Many-to-many, user-authored. “Hackathon”, “Onam”, “move-in”. Captures intent that no source carries. Low cost, and he already demonstrated the need.
Merchant	Entity — later	Real identity resolution matters once SMS and statements disagree on the same merchant string. Today a text candidate on the transaction is adequate. Promote it in the SMS part, not before.
Confidence	Two fields, not one	Currently conflated. <i>Extraction</i> confidence (did we read the number correctly — matters for PDF) and <i>classification</i> confidence (did we categorise it correctly) are independent and route to different review queues.
Reconciliation link	Field on observation	Match method, match score, and — for review — the candidate it was matched against. Not a separate entity.
Review	Entity — exists	Fine. Needs to actually be reachable.
Budget	Entity — exists, thin	Needs a period, and needs to read the budget-eligible total rather than the raw category total.

Cash flow	Derived	A query over transactions plus recurring series. Not stored.
Insight	Derived view, not stored	Storing insights guarantees staleness. Compute on read. Store only user interaction with them (dismissed, acted on) if and when that becomes a feature.
Goal	Do not build	Nothing in the workbook or the product thesis needs it. It is the most commonly built and least used feature in this category.
Benchmark	Do not build	See section 2. The only defensible benchmark is the user's own history, which needs no entity.



Deduplication attaches an observation rather than discarding a row — one mechanism covers provenance, overlapping statements, conflicts and multi-source confirmation.

4 • Three-source ingestion and reconciliation

The current pipeline does not support this, and not because of a missing feature — because of a modelling choice.

`create_transaction` deduplicates on `(user_id, source, external_id)`. The source is *part of the key*, so the same purchase seen by SMS and by a statement can never collide by construction. Cross-source dedupe is not weak in the current code; it is impossible. That is the change: match on the event, then attach the observation.

Matching rule

Match candidates within one account on: **exact signed amount** (never fuzzy-match money), **date within a window** (± 1 day for UPI, ± 3 for cards, where authorisation and settlement differ), then merchant/description similarity as a tiebreak. Amount exactness is what keeps this deterministic and testable.

Decision table

Situation	Action	Reasoning
Sources agree	Merge; attach both observations; no new transaction	Field precedence: statement/AA over SMS. The statement is the record; SMS is a notification.
Amounts differ, otherwise matching	Do not auto-merge. Link as possible-duplicate, route to review	This is authorisation vs settlement — fuel pre-auth, tips, forex. Your own workbook has the case: ₹974.71 “petrol, full tank”. Auto-merging here silently loses or invents money.
Only one source	Normal; record coverage	Not an error. The user should be able to see it is single-sourced if they ask, not be told constantly.
SMS arrives first	Create PROVISIONAL transaction, counted but flagged	Real-time value is the entire point of SMS. Withholding it until a statement confirms throws that away.
Statement arrives later	Attach observation; promote to SETTLED ; statement wins on amount and date	Keep the SMS timestamp — it is the better answer to “when did I actually spend this”.
Overlapping statements	Content hash matches; attach observation; no new transaction	The same mechanism as cross-source. This is why observations pay for themselves.
Missing from one source	Only flag if that source's coverage window <i>should</i> include it	An SMS with no matching statement line inside a covered window is a genuine signal — a reversal, a failed payment, or a non-bank rail. Outside coverage it means nothing.
Refund / reversal	Link to the original debit; net it	Not on your list, and it will bite. An unlinked refund inflates income and corrupts every category total.

One rule underneath all of it: **reconciliation may merge observations, and may never delete or alter a transaction's amount without a review record.** Anything automatic that changes a number is a bug waiting to be untraceable.

5 • Finvu

I recommended OneMoney and you have ruled it out. I accept that — but the decision has an engineering consequence worth stating plainly: **Finvu is a direct ReBIT integration, not a Setu-style managed one, and it is materially more**

work than the current abstraction assumes. That strengthens the case for putting AA last rather than weakening it.

Where the current abstraction breaks

Gap	Detail
Consent handle vs consent ID	The hard break. Setu returns an <code>id</code> and a hosted <code>url</code> immediately, and <code>app/api/aa.py</code> raises 502 if either is absent. ReBIT returns a <code>consentHandle</code> ; the consent ID only exists after approval, retrieved via <code>GET /Consent/handle/{handle}</code> . The interface has nowhere to put a handle. <code>create_consent</code> must return an opaque provider reference plus a journey URL that may be <i>constructed</i> rather than returned.
ReBIT encryption	The largest single item. <code>GET /FI/fetch/{sessionId}</code> returns encrypted FI data — per-session ECDH key material and AES-GCM, with your public key submitted in <code>POST /FI/request</code> . Nothing in the repository does any of this; Setu's Bridge hands back plaintext. The <code>fetch_data → normalize_response</code> boundary assumes plaintext JSON and needs a decryption stage between them, plus key generation and lifecycle.
Detached JWS signing	Every request carries <code>x-jws-signature</code> over the body. No signing hook exists. Needs to be a provider capability with key generation, registration with Finvu, and rotation.
Notification topology	Finvu calls three separate endpoints — consent, FI, account-link. The current design has one webhook route with a Setu event enum. Account-link notifications have no analogue in the model at all.
Inbound authentication	Finvu calls <i>your</i> public root with a token you generate and share. The endpoint that will receive this is the one that currently performs no authentication whatsoever. The webhook security finding is no longer just a security issue — it is a Finvu integration blocker.
Onboarding is manual	Root endpoint and public key by email to Cookiejar; sandbox tokens expire at six months. Build the expiry into your operational calendar now, because a silent expiry mid-development will look like a code bug.

What must stay provider-specific, and what must not

Inside the adapter: endpoint paths and base URLs; the auth scheme; request signing; the consent request payload shape; the handle/ID two-step; encryption and key registration; FI request/fetch semantics; notification payload shapes and endpoint topology; provider error codes.

Outside, and provider-neutral: consent lifecycle *states*; the data-session concept; the normalised transaction output; the ingestion path; idempotency; ownership; the observation model. The existing `AAPProvider` methods are the right shape — they need `handle_callback`, `handle_webhook`, `normalize_response` added, and a place for crypto.

Also: stop using the provider name as the source type. "SETU" is currently a `SourceType` literal, a CHECK constraint value, a hardcoded string in four repository methods, and one third of the dedupe key. Introducing "FINVU" as a new source type re-imports every existing transaction as new. The source type is `AA`; the provider belongs in the `provider` column that already exists.

Define “done” honestly

Developer/UAT integration can be completed and is worth completing: consent request, journey, approval, notification, FI request, fetch, decrypt, normalise, canonical transactions, aggregation. Against Finvu's simulated FIP this is fully testable.

Production onboarding — business agreement, Sahamati certification, central registry, and the regulated-entity licence — is not an engineering task and cannot be completed by writing code. Keep it as a parallel business track and never let a roadmap item depend on it. *Part 7 is done when the adapter passes contract tests against a local simulator, one full*

journey is verified on Finvu UAT, and a second provider can be stubbed behind the same interface. Not when AA works in production.

6 · Production readiness

Must have now — before any new feature

- Aggregation computed from canonical transactions, not from a stored snapshot blob (see section 10 — the fix is structural, not a patch)
- The observation model and content-based matching, replacing source-scoped external-ID dedupe
- A reachable review path — ambiguous classification must actually stop somewhere
- Asynchronous ingestion on a worker; ingestion currently runs inline on the event loop
- Webhook authentication — now also a Finvu blocker
- `FORCE ROW LEVEL SECURITY`, a non-owning database role, and one cross-tenant negative test against real PostgreSQL
- Version control, and rotation of the credentials currently sitting in `.env`
- An honest privacy statement (section 9)

Should have before beta

- One authenticated frontend; retire the other
- Coverage windows, and every rate rendered with its denominator
- Decimal money end-to-end; timezone-aware timestamps anchored to IST
- PDF worker isolation, parser confidence, balance-continuity validation
- Recurring series detection
- User-defined categories and tags; the `account` entity actually populated
- Account deletion actually implemented — `DELETE /api/auth/account` returns 501, which is untenable for a privacy-first product
- Rate limiting and structured audit on ingestion and webhook endpoints

Can wait

- Merchant identity resolution · goals · benchmarks (or never) · multi-currency · LLM insight layer · AA production access · automatic cross-account transfer detection (needs two connected accounts to be worth anything)

7 · Roadmap

I disagree with the current order in four places, and each is a dependency problem rather than a preference.

- **PDF (3) before review (4) is inverted.** PDF is the source most likely to produce uncertain rows. It needs the review path to exist first.
- **SMS (6) is behind PDF (3), and should be ahead of it.** SMS is the easiest source to build, delivers the highest immediate user value, and — per Discovery 2 — is the only source that captures the micro-transactions dominating actual behaviour. It is also the cheapest possible test of the observation model before PDF's complexity lands on top.
- **Frontend (9) is last, so nothing is ever validated in production auth mode.** Since `ENVIRONMENT=production` forces authentication and neither frontend sends a token, Parts 3–8 will all be built and tested in a configuration production never uses.
- **Security (10) is last, but two of its items are blocking now.** Webhook auth blocks Finvu; RLS blocks real user data.

Part	Proposed	Why here
3	Correctness & convergence core — aggregate from transactions, observation model, matching and reconciliation engine, reachable review, async worker, coverage windows	No new source. Everything downstream needs all of it, and it is the cheapest it will ever be to build.
4	Security & privacy floor — webhook auth, forced RLS with a test, real deletion, honest privacy statement, git	Small, blocking, and two items gate Finvu and real users respectively.
5	Android SMS	Easiest source, highest value, exercises reconciliation with a real latency gap before PDF's difficulty arrives.
6	PDF/CSV statements — isolation, bank parsers, financial validation	Now review, dedupe, worker and coverage all exist for it to land on.
7	One authenticated frontend — overview, drill-down, review queue, coverage	Moves up from 9. Everything after this is validated in the real configuration.
8	Deterministic intelligence — recurring series, decomposition, exceptional attribution, counterfactual savings, budgets	This is the spreadsheet, delivered. All arithmetic, no LLM.
9	Finvu AA behind the adapter, verified on UAT and a local simulator	Lowest marginal information, highest cost, external blocker. Last of the sources.
10	LLM layer and hardening	Only meaningful once there is correct data to describe.

Net effect: correctness and reconciliation move ahead of every new source, SMS overtakes PDF, the frontend moves from last to middle, and AA moves from seventh to ninth.

8 · Frontend

The organising idea: **the Insights sheet is the product, and every sentence in it should be a clickable object.** Not “here is a pie chart”, but “here is a claim about your money, here is the arithmetic, here are the transactions it rests on.”

Screen	What it answers
Now	Spend so far this period against a typical period, with a coverage banner: “statements through 31 Aug, SMS through today, 4 days unobserved.” The banner is not a warning — it is the honest frame for every number below it.
Why	The headline decomposed: recurring baseline, one-off, exceptional. Each expandable to the transactions. This screen is the product. Nothing in either existing frontend resembles it.
Recurring	Detected series with cadence, typical amount and next expected date. “ChatGPT Go, monthly, ₹400, next 18 Sep.” Alerts when an amount changes or an expected charge does not arrive.
Review	The only screen with a to-do: ambiguous classification, possible duplicates, low-confidence extractions, source conflicts. Designed to be empty most of the time. If it is never empty, the classifier is wrong.
Transactions	The ledger. Fast search, edit, tag, split. Source shown on demand, not as a permanent badge.
Sources	What is connected, what window each covers, what is stale. This replaces the current privacy screen, which asserts things that are no longer true.
Capture	Sub-five-second cash entry. Given that 87 of 93 transactions are ₹20–₹110 daily items, this may be the most-used screen in the product.

Two rules worth enforcing globally: **every number is traceable to its transactions in one click**, and **every rate displays its denominator**.

9 · AI and financial intelligence

One hard rule, and it settles most of the question: **the LLM may read computed aggregates and write English. It may never produce a number that deterministic code did not compute.**

Deterministic, always	AI-assisted, always reviewable	Never an LLM
All arithmetic and totals · rule-matched categorisation · duplicate matching · transfer detection · recurring detection · budget computation · balance validation · coverage · the decision that something needs review	Merchant string → human-readable name · first-pass category for genuinely novel descriptions (the existing embedding classifier is better suited than an LLM here) · natural-language question → structured query over aggregates · prose rendering of already-computed facts	Computing or adjusting any amount · deciding budget inclusion · deciding what is a duplicate · writing to the canonical transaction table · generating a savings figure · anything the user cannot trace back to transactions

On “Goldfish Memory” — I am reversing my previous position

My reconnaissance treated the privacy-claim mismatch as a truthfulness bug to be fixed by aligning the wording with the storage. Having read the workbook, I think that was too soft. **The concept itself is incompatible with the product.**

Every valuable output in that spreadsheet requires per-transaction retention. “The ₹1,740 on 9 August is 14% of your two months” needs the transaction. Drill-down needs the transaction. Recurring detection needs history. Cross-source deduplication needs history. Coverage needs history. A product that forgets individual transactions cannot produce the

Insights sheet — and the repository already abandoned the idea in Part 2, when the transactions table began persisting full narrations, while the README, the privacy endpoint and the frontend copy continued to advertise it.

Retire the concept and replace it with a claim that is specific, true, and still differentiating: **raw source artifacts are ephemeral — PDF bytes, PDF passwords, SMS bodies, encrypted AA payloads and decryption keys are destroyed after extraction and never enter the database. Derived canonical transactions are durable, user-owned, exportable and deletable on request.** That is a real privacy position, it survives contact with the product, and — unlike the current claim — it can be tested. Also implement deletion, which currently returns 501.

10 · The biggest architectural risk

Treating a precomputed aggregate blob as the read model instead of querying the canonical transaction table.

`aggregate_snapshots` stores a JSON document that the entire dashboard reads. That single choice is the root cause of the aggregation defect I reported — but the defect is the symptom, and fixing it without deleting the pattern leaves the real problem in place.

Consider what the workbook asks for. Drill-down from a number to its transactions: impossible from a blob. Recurring series: needs transaction history. Exceptional-spend attribution: needs the individual transaction. Coverage: needs to know which days have rows. Reconciliation: needs the ledger. Every single valuable output requires querying transactions — and none of them can be served by a document that was computed at write time.

The failure mode is gradual and expensive. Each new insight in Parts 4, 5 and 8 becomes a new field in the blob. Each requires a full re-aggregation to backfill. Reconciliation, which mutates history retroactively when a statement confirms an SMS, invalidates every snapshot after the affected date. Eventually you are maintaining a denormalised document with the schema of a reporting database and none of its guarantees, and the ledger underneath it is decorative.

The blob is a cache. It is currently pretending to be a model. **Make the transaction table the source of truth for every read, add a materialised cache later only if measurement demands it, and delete the snapshot as an authority.** Doing this in Part 3, before three sources and reconciliation land on top, is a contained change. Doing it in Part 8 is a rewrite.

A · What you are building correctly

- The canonical pipeline as a concept, and the discipline of one convergence point for all sources. Most projects in this category never get this far.
- Separating lifecycle, classification, budget inclusion, transfer and duplicate state into independent dimensions. This is the right model and it is already in the schema.
- Deriving ownership from the validated JWT and refusing a body-supplied user ID.
- Keeping provenance on every transaction — source, record ID, external ID, import ID.
- Insisting that financial calculation stays deterministic and testable. Hold that line; it is the correct instinct and it is unusual.
- Treating the AA layer as replaceable from the start. The interface needs extending, not replacing.
- Writing the spreadsheet at all. It is a better product specification than most PRDs, and the fact that its own arithmetic slipped is the argument for building the software.

B · What you are getting wrong

- **The aggregate blob as read model.** Section 10. The largest single risk.
- **Source is part of the deduplication key**, which makes cross-source reconciliation structurally impossible rather than merely absent.
- **No coverage concept**, which is how the reference workbook reached a conclusion that is the reverse of what its own data supports.
- **AA prioritised above its value.** It is the lowest-marginal-information source, the highest-cost integration, and the only one with an external blocker.
- **Cash and manual entry treated as a fallback** when the evidence says they are the primary source by transaction count.
- **A privacy claim the product has already outgrown**, still advertised in the README, the API and the UI.
- **Building dashboards when the product is an explanation engine.**
- **Planning a benchmark feature** on cost-of-living blog figures.

C · What to fix immediately

1. Make the transaction table the read model. Delete `aggregate_snapshots` as an authority; keep it only as a cache, if at all.
2. Introduce the observation entity and content-based matching. Remove source from the dedupe key.
3. Make the review path reachable — ambiguous classification must stop somewhere.
4. Move ingestion onto a worker, off the request thread.
5. Authenticate the webhook. It blocks Finvu as well as being a live exposure.
6. `FORCE ROW LEVEL SECURITY`, non-owning database role, one cross-tenant negative test on real PostgreSQL.
7. `git init`, and rotate the Setu credentials in `.env`.
8. Rewrite the privacy claim to something true, and implement deletion.

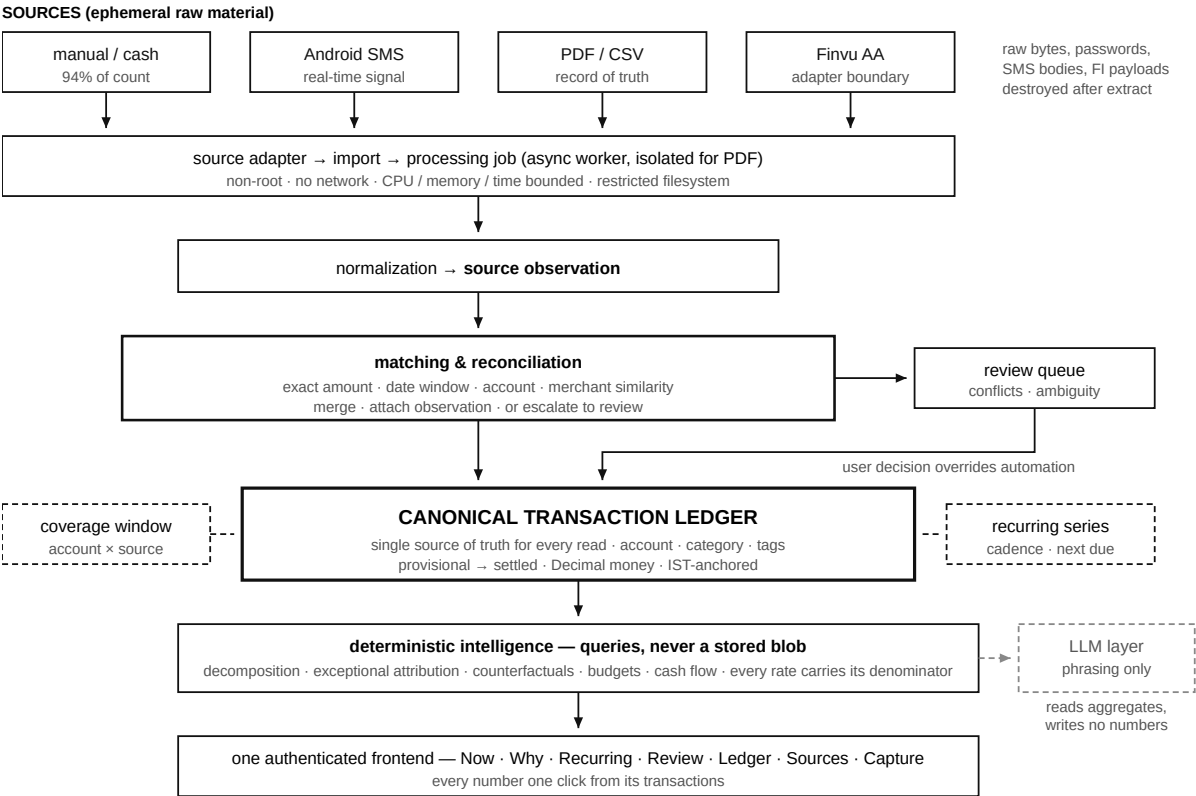
D · What to build next

1. **Coverage windows** — per account, per source. Then make every rate in the product render its denominator. Small, and it is what separates this from the spreadsheet.
2. **SMS ingestion** — the cheapest source, the highest immediate value, and the honest test of whether reconciliation works.
3. **Fast cash capture** — sub-five-second entry. On the evidence, the most-used surface in the product.
4. **Recurring series detection** — unlocks the workbook's central insight, plus forecasting and price-change alerts.
5. **The “Why” screen** — decomposition with drill-down. The first screen that is recognisably Aashan rather than a generic finance dashboard.

E · What I would not build yet

- **Goals.** Nothing needs them.
- **Benchmarks against external cohorts.** Possibly never. Compare the user to their own past instead.
- **Per-transaction multi-source confidence in the UI.** Keep the evidence internally; do not put a badge on lunch.
- **Merchant identity resolution.** Wait until two sources disagree on a merchant string and give you real training cases.
- **Any LLM feature.** Not until the deterministic numbers are correct, at which point the LLM's job will be smaller than currently imagined.
- **Multi-currency.** The schema is currency-aware; that is sufficient for years.
- **AA production onboarding as an engineering dependency.** Run it as a business track.
- **The second frontend.** Delete one.

F · Recommended architecture



Three structural changes from today: observations sit between normalization and the ledger; the ledger is the read model for everything; coverage and recurring series hang off it as first-class dimensions. Everything else in the current architecture survives.